

MODULE

08

Projet intégrateur —
UnivEvent
Licence 3 Génie Logiciel
Université Joseph Kl-
ZERBO · IFOAD

PROJET INTÉGRATEUR · 2025-2026

Modélisation des données

MCD, MLD, schéma physique et règles de gestion du Module 8

MODULE	N°08 — Gestion des retraits de diplômes
ÉTUDIANT	Boukary DIALLO
ENCADRANT	Dr KOALA
BRANCHE	feat/module-8-diploma-withdrawal
DÉPÔT	github.com/BoukaryDiallo/UnivEvent
DATE	29 avril 2026

Sommaire

01 Modélisation des données — Module 8

Sommaire
1. Entités du module
2. MCD — Diagramme entité-association
3. Règles de gestion
4. MLD — Modèle relationnel
5. Schéma physique (DDL)
6. Index et contraintes d'intégrité
Annexe — Mapping vers le code

Modélisation des données — Module 8

Ce document présente le **MCD** (Modèle Conceptuel de Données, formalisme Merise) puis le **MLD** (Modèle Logique de Données, traduction relationnelle) du module de gestion des retraits de diplômes. Les choix techniques (clés primaires, étrangères, contraintes d'intégrité, index) sont issus directement des migrations versionnées dans `database/migrations/2026_04_01_*`.

Sommaire

- 1. [Entités du module](#)
- 2. [MCD — Diagramme entité-association](#)
- 3. [Règles de gestion](#)
- 4. [MLD — Modèle relationnel](#)
- 5. [Schéma physique \(DDL\)](#)
- 6. [Index et contraintes d'intégrité](#)

1. Entités du module

ENTITÉ	RÔLE
USER	Utilisateur de la plateforme (étudiant ou agent de scolarité). L'entité est portée par le Module 1 ; le module 8 la référence uniquement.
DIPLOMA_REQUEST	Demande de retrait de diplôme. Cycle de vie en 8 états (Draft → Submitted → DocumentsValidated → ReadyForPickup → AppointmentScheduled → Delivered → Archived , plus l'état terminal Rejected).
DIPLOMA_DOCUMENT	Pièce justificative téléversée par l'étudiant et liée à une demande (CNI, relevé de notes, attestation de succès, photo, reçu de paiement, autre).
DIPLOMA_REQUEST_EVENT	Évènement d'audit consignnant chaque transition d'état d'une demande, avec auteur et note.
PICKUP_SLOT	Créneau de retrait planifié par la scolarité (date, plage horaire, lieu, capacité).
PICKUP_APPOINTMENT	Rendez-vous concret réservé par un étudiant sur un créneau, pour une demande donnée.

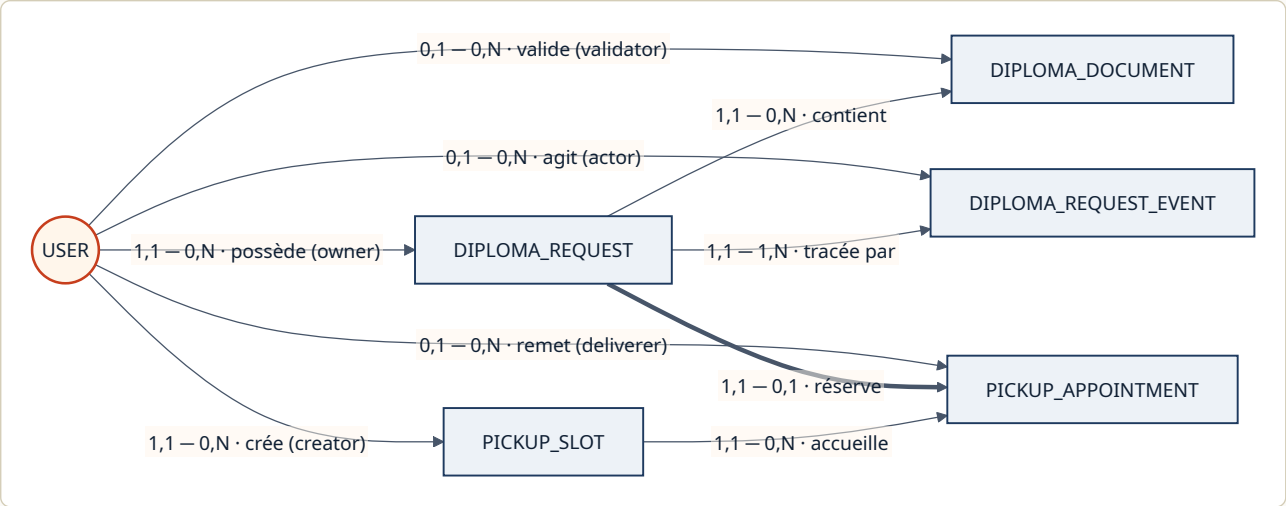
2. MCD — Diagramme entité-association

2.1 NOTATION

Pour chaque association on lit la cardinalité au sens Merise :

- `(0, 1)` : zéro ou une fois.
- `(1, 1)` : une et une seule fois (obligatoire).
- `(0, N)` : zéro ou plusieurs fois.
- `(1, N)` : au moins une fois.

2.2 DIAGRAMME



Les attributs détaillés de chaque entité sont consignés dans le **dictionnaire de données** (section 4.3) et reproduits dans le DDL (section 5).

2.3 CARDINALITÉS RÉSUMÉES

ASSOCIATION	CÔTÉ GAUCHE	CÔTÉ DROIT
USER possède DIPLOMA_REQUEST	(1,1)	(0,N)
USER agit sur DIPLOMA_REQUEST_EVENT	(0,1)	(0,N)
USER valide DIPLOMA_DOCUMENT	(0,1)	(0,N)
USER crée PICKUP_SLOT	(1,1)	(0,N)
USER remet PICKUP_APPOINTMENT	(0,1)	(0,N)
DIPLOMA_REQUEST contient DIPLOMA_DOCUMENT	(1,1)	(0,N)
DIPLOMA_REQUEST tracée par DIPLOMA_REQUEST_EVENT	(1,1)	(1,N)
DIPLOMA_REQUEST réserve PICKUP_APPOINTMENT	(1,1)	(0,1)
PICKUP_SLOT accueille PICKUP_APPOINTMENT	(1,1)	(0,N)

3. Règles de gestion

Les règles métier non triviales, exprimées au niveau conceptuel, sont :

- **RG1 — Code de suivi unique.** Chaque `DIPLOMA_REQUEST` reçoit à la création un code `DIP-AAAA-XXXXXXX` unique. La génération est protégée par retry-on-collision dans le service.
- **RG2 — Soumission gardée.** Une demande ne peut passer de `Draft` à `Submitted` que si elle contient au moins une pièce.
- **RG3 — Validation gardée.** Une demande ne peut passer de `Submitted` à `DocumentsValidated` que si **toutes** ses pièces ont `validated_at` $\neq$ null .
- **RG4 — Un seul RDV par demande.** À tout instant, une `DIPLOMA_REQUEST` est associée à **0 ou 1** `PICKUP_APPOINTMENT` . La contrainte est portée par une clé UNIQUE sur `pickup_appointments.diploma_request_id` .
- **RG5 — Pas de chevauchement de créneaux par lieu.** Deux `PICKUP_SLOT` au même `location` ne peuvent pas se recouvrir temporellement (vérification applicative).
- **RG6 — Capacité respectée.** Le nombre de `PICKUP_APPOINTMENT` rattachés à un `PICKUP_SLOT` ne peut excéder sa `capacity` (vérification dans une transaction avec `lockForUpdate`).
- **RG7 — Audit complet.** Toute transition d'état d'une `DIPLOMA_REQUEST` génère **un et un seul** `DIPLOMA_REQUEST_EVENT` (paire `from_status` / `to_status`).
- **RG8 — Annulation libère la place.** L'annulation d'un RDV supprime la ligne `PICKUP_APPOINTMENT` et rebascule la demande à `ReadyForPickup` , libérant la capacité du créneau.

- **RG9 — Suppression d'un brouillon.** Une `DIPLOMA_REQUEST` ne peut être supprimée que dans l'état `Draft` (soft delete via `deleted_at` pour conserver l'historique).
- **RG10 — Statut terminal.** Les états `Delivered`, `Archived` et `Rejected` n'autorisent aucune transition supplémentaire.

4. MLD — Modèle relationnel

Traduction du MCD en tables relationnelles. Pour chaque association `1,1 - 0,N` ou `1,1 - 0,1`, on porte la clé étrangère côté N (ou côté optionnel `0,1`).

4.1 LISTE DES RELATIONS

```
users (id, name, email, est_actif, password, email_verified_at,
      remember_token, two_factor_*, created_at, updated_at)

diploma_requests (id, #owner_id, tracking_code, diploma_type,
                  academic_year, status, submitted_at, rejected_reason,
                  archived_at, created_at, updated_at, deleted_at)

diploma_documents (id, #diploma_request_id, type, path, original_name,
                  mime, size, validated_at, #validated_by,
                  created_at, updated_at)

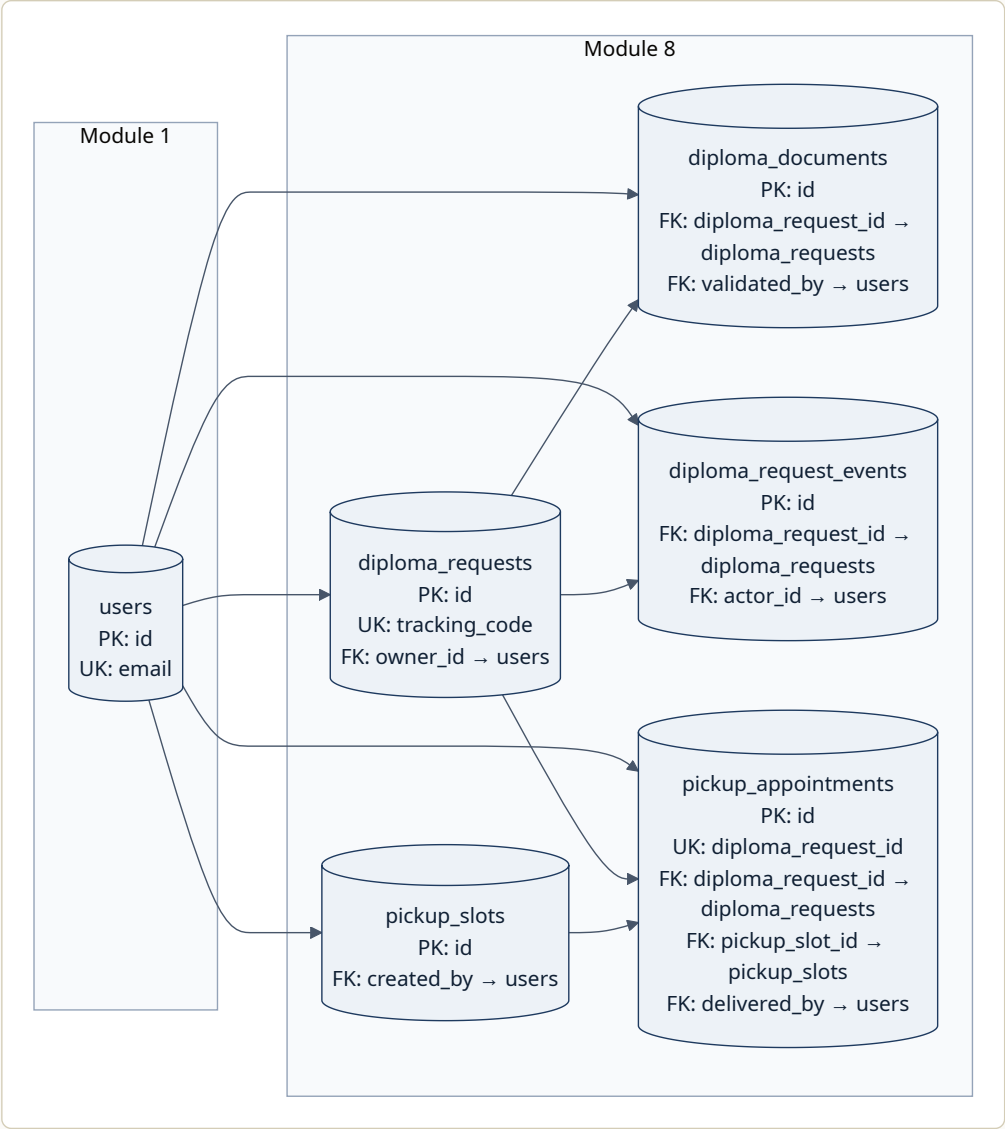
diploma_request_events (id, #diploma_request_id, from_status, to_status,
                       #actor_id, note, occurred_at,
                       created_at, updated_at)

pickup_slots (id, starts_at, ends_at, capacity, location,
              #created_by, created_at, updated_at)

pickup_appointments (id, #diploma_request_id*, #pickup_slot_id,
                    confirmed_at, delivered_at, #delivered_by,
                    receipt_path, created_at, updated_at)
```

Légende: `id` = clé primaire — `#xxx` = clé étrangère — `*` indique une contrainte UNIQUE.

4.2 DIAGRAMME DU MLD



4.3 DICTIONNAIRE DE DONNÉES

diploma_requests

COLONNE	TYPE	NULLABLE	CONTRAINTE	DESCRIPTION
id	bigint unsigned	non	PK	Identifiant interne
owner_id	bigint unsigned	non	FK → users(id) ON DELETE CASCADE	Étudiant propriétaire
tracking_code	varchar(20)	non	UNIQUE	Code de suivi <code>DIP-AAAA-XXXXXXX</code>
diploma_type	varchar(50)	non		<code>licence</code> , <code>master</code> ou <code>doctorat</code>
academic_year	varchar(9)	non		Format <code>AAAA-AAAA</code> , années courantes (Rule::in)
status	varchar(32)	non	défaut <code>draft</code>	Valeur de l'enum <code>DiplomaRequestStatus</code>
submitted_at	timestamp	oui		Date/heure de soumission
rejected_reason	text	oui		Motif si statut <code>rejected</code>
archived_at	timestamp	oui		Date/heure d'archivage
created_at, updated_at	timestamp	non		Horodatage Eloquent
deleted_at	timestamp	oui		Soft delete

diploma_documents

COLONNE	TYPE	NULLABLE	CONTRAINTE	DESCRIPTION
id	bigint unsigned	non	PK	
diploma_request_id	bigint unsigned	non	FK → diploma_requests(id) ON DELETE CASCADE	Demande propriétaire
type	varchar(40)	non		Valeur de <code>DocumentType</code>
path	varchar(255)	non		Chemin sur le disque privé
original_name	varchar(255)	non		Nom du fichier au moment du dépôt
mime	varchar(100)	non		MIME sniffé via <code>finfo</code>
size	bigint unsigned	non		Taille en octets
validated_at	timestamp	oui		Date de validation
validated_by	bigint unsigned	oui	FK → users(id) ON DELETE SET NULL	Agent qui a validé
created_at, updated_at	timestamp	non		

diploma_request_events

COLONNE	TYPE	NULLABLE	CONTRAINTE	DESCRIPTION
id	bigint unsigned	non	PK	
diploma_request_id	bigint unsigned	non	FK → diploma_requests(id) ON DELETE CASCADE	
from_status	varchar(32)	oui		État avant la transition (null à la création)
to_status	varchar(32)	non		État après la transition
actor_id	bigint unsigned	oui	FK → users(id) ON DELETE SET NULL	Auteur de la transition
note	text	oui		Commentaire libre (motif de rejet, etc.)
occurred_at	timestamp	non		Horodatage de la transition
created_at, updated_at	timestamp	non		

pickup_slots

COLONNE	TYPE	NULLABLE	CONTRAINTE	DESCRIPTION
id	bigint unsigned	non	PK	
starts_at	timestamp	non		Début du créneau
ends_at	timestamp	non		Fin (> starts_at)
capacity	int unsigned	non		1 à 200
location	varchar(255)	non		Salle / lieu de retrait
created_by	bigint unsigned	non	FK → users(id)	Agent qui a planifié
created_at, updated_at	timestamp	non		

pickup_appointments

COLONNE	TYPE	NULLABLE	CONTRAINTE	DESCRIPTION
id	bigint unsigned	non	PK	
diploma_request_id	bigint unsigned	non	UNIQUE , FK → diploma_requests(id) ON DELETE CASCADE	Une seule réservation par demande
pickup_slot_id	bigint unsigned	non	FK → pickup_slots(id) ON DELETE CASCADE	
confirmed_at	timestamp	non		Date de la réservation
delivered_at	timestamp	oui		Acté lors de la remise
delivered_by	bigint unsigned	oui	FK → users(id) ON DELETE SET NULL	
receipt_path	varchar(255)	oui		Reçu signé (optionnel)
created_at, updated_at	timestamp	non		

5. Schéma physique (DDL)

Extraits SQL effectivement émis par les migrations Laravel (compatibles MySQL 8 / PostgreSQL 14 / SQLite 3.40).

```

CREATE TABLE diploma_requests (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  owner_id    BIGINT UNSIGNED NOT NULL,
  tracking_code VARCHAR(20) NOT NULL,
  diploma_type VARCHAR(50) NOT NULL,
  academic_year VARCHAR(9) NOT NULL,
  status      VARCHAR(32) NOT NULL DEFAULT 'draft',
  submitted_at TIMESTAMP NULL,
  rejected_reason TEXT NULL,
  archived_at  TIMESTAMP NULL,
  created_at   TIMESTAMP NULL,
  updated_at   TIMESTAMP NULL,
  deleted_at   TIMESTAMP NULL,

  UNIQUE KEY ux_diploma_requests_tracking_code (tracking_code),
  KEY idx_diploma_requests_status_submitted_at (status, submitted_at),
  KEY idx_diploma_requests_owner_id (owner_id),
  CONSTRAINT fk_diploma_requests_owner FOREIGN KEY (owner_id)
    REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE diploma_documents (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  diploma_request_id BIGINT UNSIGNED NOT NULL,
  type        VARCHAR(40) NOT NULL,
  path        VARCHAR(255) NOT NULL,
  original_name VARCHAR(255) NOT NULL,
  mime        VARCHAR(100) NOT NULL,
  size        BIGINT UNSIGNED NOT NULL,
  validated_at  TIMESTAMP NULL,
  validated_by  BIGINT UNSIGNED NULL,
  created_at   TIMESTAMP NULL,
  updated_at   TIMESTAMP NULL,

  KEY idx_diploma_documents_request_type (diploma_request_id, type),
  CONSTRAINT fk_diploma_documents_request FOREIGN KEY (diploma_request_id)
    REFERENCES diploma_requests(id) ON DELETE CASCADE,
  CONSTRAINT fk_diploma_documents_validator FOREIGN KEY (validated_by)
    REFERENCES users(id) ON DELETE SET NULL
);

CREATE TABLE diploma_request_events (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  diploma_request_id BIGINT UNSIGNED NOT NULL,
  from_status  VARCHAR(32) NULL,
  to_status    VARCHAR(32) NOT NULL,
  actor_id     BIGINT UNSIGNED NULL,
  note         TEXT NULL,
  occurred_at  TIMESTAMP NOT NULL,
  created_at   TIMESTAMP NULL,
  updated_at   TIMESTAMP NULL,

  KEY idx_diploma_events_request_occurred (diploma_request_id, occurred_at),
  CONSTRAINT fk_diploma_events_request FOREIGN KEY (diploma_request_id)
    REFERENCES diploma_requests(id) ON DELETE CASCADE,
  CONSTRAINT fk_diploma_events_actor FOREIGN KEY (actor_id)
    REFERENCES users(id) ON DELETE SET NULL
);

CREATE TABLE pickup_slots (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  starts_at   TIMESTAMP NOT NULL,
  ends_at     TIMESTAMP NOT NULL,
  capacity    INT UNSIGNED NOT NULL,
  location    VARCHAR(255) NOT NULL,
  created_by  BIGINT UNSIGNED NOT NULL,
  created_at  TIMESTAMP NULL,
  updated_at  TIMESTAMP NULL,

  CONSTRAINT fk_pickup_slots_creator FOREIGN KEY (created_by)
    REFERENCES users(id)
);

CREATE TABLE pickup_appointments (

```

```

    id                BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    diploma_request_id BIGINT UNSIGNED NOT NULL,
    pickup_slot_id     BIGINT UNSIGNED NOT NULL,
    confirmed_at        TIMESTAMP NOT NULL,
    delivered_at        TIMESTAMP NULL,
    delivered_by        BIGINT UNSIGNED NULL,
    receipt_path        VARCHAR(255) NULL,
    created_at          TIMESTAMP NULL,
    updated_at          TIMESTAMP NULL,

    UNIQUE KEY ux_pickup_appointments_request (diploma_request_id),
    KEY idx_pickup_appointments_slot (pickup_slot_id),
    CONSTRAINT fk_pickup_appointments_request FOREIGN KEY (diploma_request_id)
        REFERENCES diploma_requests(id) ON DELETE CASCADE,
    CONSTRAINT fk_pickup_appointments_slot FOREIGN KEY (pickup_slot_id)
        REFERENCES pickup_slots(id) ON DELETE CASCADE,
    CONSTRAINT fk_pickup_appointments_deliverer FOREIGN KEY (delivered_by)
        REFERENCES users(id) ON DELETE SET NULL
);

```

6. Index et contraintes d'intégrité

6.1 INDEX APPLICATIFS

TABLE	INDEX	JUSTIFICATION
diploma_requests	(status, submitted_at)	Requête principale du back-office : « toutes les demandes soumises triées par date ».
diploma_requests	(owner_id)	Listing des demandes d'un étudiant sur son tableau de bord.
diploma_documents	(diploma_request_id, type)	Lookup des pièces d'un dossier ; permet aussi un filtrage rapide par type.
diploma_request_events	(diploma_request_id, occurred_at)	Reconstruction de la timeline d'une demande, ordonnée.
pickup_appointments	pickup_slot_id	Comptage des RDV d'un créneau pour vérifier la capacité.

6.2 CONTRAINTES D'INTÉGRITÉ RÉFÉRENTIELLE

RELATION	ACTION ON DELETE	JUSTIFICATION
diploma_requests.owner_id → users.id	CASCADE	Si un compte est purgé, ses dossiers le sont aussi (purgable même via soft delete grâce à deleted_at).
diploma_documents.diploma_request_id → diploma_requests.id	CASCADE	Une pièce n'a aucun sens sans son dossier.
diploma_documents.validated_by → users.id	SET NULL	Si un agent quitte l'institution, on garde la pièce mais on perd la trace de qui l'a validée.
diploma_request_events.diploma_request_id → diploma_requests.id	CASCADE	Audit lié au cycle de vie.
diploma_request_events.actor_id → users.id	SET NULL	Idem : on conserve l'évènement, on perd l'auteur.
pickup_slots.created_by → users.id	RESTRICT (par défaut)	On ne supprime pas un agent ayant des créneaux actifs.
pickup_appointments.diploma_request_id → diploma_requests.id	CASCADE	Si la demande est purgée, le RDV n'a plus de sens.
pickup_appointments.pickup_slot_id → pickup_slots.id	CASCADE	Idem.
pickup_appointments.delivered_by → users.id	SET NULL	Conserver la trace de la remise même si l'agent disparaît.

6.3 CONTRAINTES D'UNICITÉ

- diploma_requests.tracking_code : UNIQUE (RGI).

- `pickup_appointments.diploma_request_id` : UNIQUE (RG4) — empêche au niveau base la création d'un second RDV sur la même demande, même en cas de race condition côté applicatif.
- `users.email` : UNIQUE (Module 1).

6.4 CONTRAINTES APPLICATIVES NON TRIVIALES

Les règles suivantes ne sont pas exprimables en SQL standard et sont portées par les services + les FormRequest + les tests :

- Validation du format `tracking_code` (regex `DIP-[d{4}]-[A-Z0-9]{8}`).
- Validation de `academic_year` contre la liste dynamique `App\Support\AcademicYear::values()` .
- Vérification du chevauchement de créneaux par lieu (`PickupService::assertNoOverlap`).
- Vérification de capacité avec verrou pessimiste (`PickupService::bookSlot`).
- Vérification que toutes les pièces sont validées avant `DocumentsValidated` .

Annexe — Mapping vers le code

CONCEPT MCD	MIGRATION	MODÈLE ELOQUENT	ENUM
DIPLOMA_REQUEST	<code>2026_04_01_091352_create_diploma_requests_table.php</code>	DiplomaRequest	DiplomaRequestStatus
DIPLOMA_DOCUMENT	<code>2026_04_01_094218_create_diploma_documents_table.php</code>	DiplomaDocument	DocumentType
DIPLOMA_REQUEST_EVENT	<code>2026_04_01_101547_create_diploma_request_events_table.php</code>	DiplomaRequestEvent	—
PICKUP_SLOT	<code>2026_04_01_143809_create_pickup_slots_table.php</code>	PickupSlot	—
PICKUP_APPOINTMENT	<code>2026_04_01_150126_create_pickup_appointments_table.php</code>	PickupAppointment	—